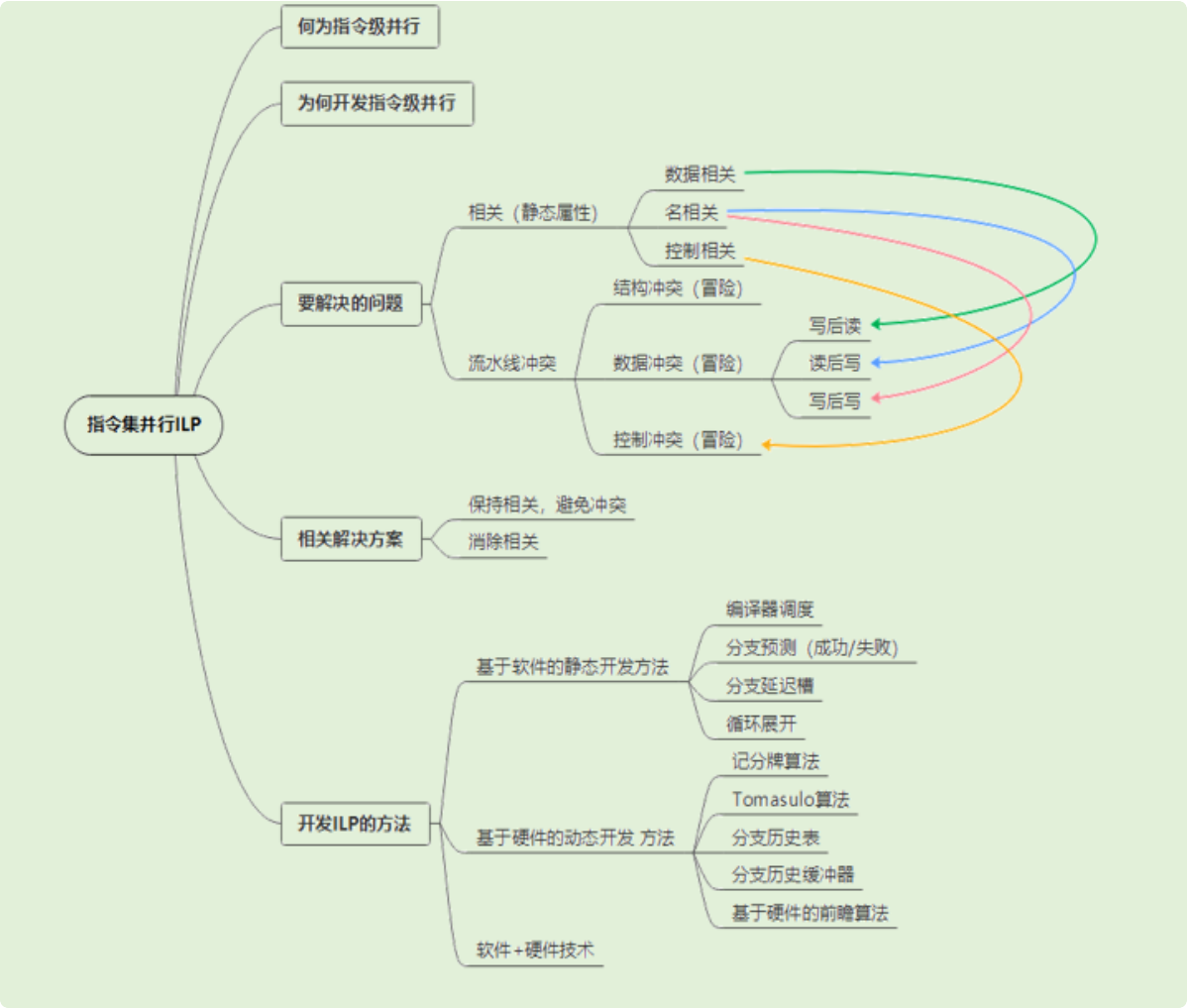
第3章 指令级并行（ILP）

一、核心整体框架



二、指令级并行基础概念

2.1 ILP定义与特点

指令级并行（ILP）：**指令一级的并行性**，特指**机器语言层面**，存储器访问、整型、浮点等不同类型指令之间可重叠执行的并行性。

核心特点：并行性**由处理器硬件+编译程序自动识别利用，无需程序员修改顺序程序代码**，与处理器迭代发展深度绑定。

2.2 核心性能公式（必考）

- **CPI（Cycles Per Instruction，每条指令的时钟周期数）**：执行一条指令平均需要多少个时钟周期。CPI 越低，说明指令执行效率越高。
- **IPC（Instructions Per Cycle，每个时钟周期完成的指令数）**：每个时钟周期能完成多少条指令。它和 CPI 是**倒数关系**：

  $$\text{IPC} = \frac{1}{\text{CPI}}$$

  所以，CPI 越低，IPC 越高，处理器的性能（单位时间能完成的指令数）就越强。

流水线实际CPI计算公式：

$$\text{CPI}_{\text{流水线}} = \text{CPI}_{\text{理想}} + \text{停顿}_{\text{结构冲突}} + \text{停顿}_{\text{数据冲突}} + \text{停顿}_{\text{控制冲突}}$$

优化核心：减少公式中各类停顿周期，降低整体CPI，提升IPC。

理想CPI：无任何冲突、无停顿的流水线最优性能指标。

2.3 程序三大固有相关（静态属性）

相关是程序天生自带的依赖关系，是导致流水线冒险的根本原因，分为三类：

1. 数据相关（真相关，无法消除）

定义：两条指令访问同一寄存器/存储单元，且存在**先写后读**的依赖关系。

本质：指令间存在真实的数据依赖，只能避让、无法消除，会**引发写后读（RAW）数据冒险。**

示例：

```
L.D     F0, 0(R1)
ADD.D   F4, F0, F2
```

2. 名相关（假相关，可消除）

定义：两条指令使用相同的寄存器名/存储地址，但无真实数据传递，仅名字冲突。可通过**寄存器重命名**彻底消除。

反相关（读后写 WAR）：后指令写、前指令读同一寄存器**（先读后写）**

输出相关（写后写 WAW）：两条指令先后写同一寄存器**（先写后写）**

3. 控制相关

定义：指令执行顺序依赖**分支指令**的执行结果，分支跳转与否决定后续指令是否执行。是**控制冒险的根本成因**。

2.4 三大流水线冒险（动态冲突）

冒险（冲突）：流水线执行中，因指令相关、资源不足，导致下一条指令无法在指定周期执行的现象，是流水线的动态属性。

冒险类型	子类型	核心成因	核心解决方案
结构冒险	—	硬件资源不足，指令争抢同一部件 / 总线	增设硬件资源、流水线阻塞
数据冒险	写后读 (RAW)	真数据相关，先写后读，有真实依赖	定向旁路、指令调度
	读后写 (WAR)	名相关（反相关），仅寄存器名冲突	寄存器重命名
	写后写 (WAW)	名相关（输出相关），多指令写同名寄存器	寄存器重命名
控制冒险	—	分支 / 跳转改变执行流程，流水线取指错误	静态分支预测（预测分支失败/成功）、 延迟分支（延迟槽）、前瞻执行

2.5 相关与冒险的核心区别

1. 相关：**程序静态固有属性**，永久存在，不会随流水线改变；

2. 冒险：**流水线动态属性**，相同相关在不同流水线中，可能产生停顿，也可能不产生，即使产生停顿停顿周期也不一定相同。

2.6 相关的两类解决思路

1. **保持相关、避免冲突：编译器静态指令调度，拉开相关指令间距，**规避停顿；

2. **代码变换、消除相关：寄存器重命名，彻底消除WAR、WAW名相关。**

三、基于软件的静态ILP开发技术（编译器层面）

核心：**编译阶段**优化代码顺序、结构，挖掘指令并行性，无需硬件参与。核心技术：**指令调度、循环展开。**

编译器调度的能力受两个特性限制：

程序固有的指令级并行性

简单说：程序里到底有多少不相关的指令。如果程序本身数据依赖极强（比如每一条指令都要等前一条的结果），就算展开 + 调度，也找不到足够的无关指令来并行，CPI 还是降不下来。

流水线功能部件的执行延迟

比如浮点加法指令要 3 个周期才能出结果，就算有无关指令，也必须满足这个延迟，不能随便调度。比如 L.D 指令执行完，必须等 1 个周期才能用它的结果做加法，这个延迟是硬件定死的，编译器没法突破。

3.1 指令调度

原理：编译器调整指令执行顺序，将无数据相关的指令穿插在阻塞空转周期中，填充流水线空隙，减少停顿。

本节中，我们使用的浮点流水线延迟为：

产生结果的指令	使用结果的指令	延迟（时钟周期数）
浮点计算	另一个浮点计算	3
浮点计算	浮点store（S.D）	2
浮点load（L.D）	浮点计算	1
浮点load（L.D）	浮点store（S.D）	0

1. 整数指令的流水线：5 段单周期，“跑起来飞快”

你之前学的 MIPS 5 段流水线（IF→ID→EX→MEM→WB），**每一段只需要 1 个时钟周期**，而且整数运算（ADD、DADDIU 等）的 `EX` 阶段（执行阶段）1 个周期就能出结果。

更关键的是：PPT 里假设「整数运算部件是全流水 / 资源足够」，意思是：

- 多个整数运算指令可以同时在流水线的不同段执行，**不会抢硬件资源（无结构冲突）**；
- 整数指令的结果出得极快，大部分数据相关都能被「定向技术」解决，几乎没有额外停顿。

所以整数指令的延迟规则非常简单，只有两种情况会停顿：

- 整数 load（LW/LD）后面跟整数运算 / 分支指令：延迟 = 1 个周期（因为 load 的结果要到 `MEM` 阶段才出来，运算 / 分支在 `ID` 阶段就要读，必须等 1 个周期）；
- 分支指令（BNE/BEQZ）：固定延迟 = 1 个周期（控制冲突停顿）。

3.2 循环展开

原理：复制循环体代码、修改循环下标与终止条件、删除冗余循环控制指令，扩大指令调度空间，开发**循环级并行性**。

配套优化：必须结合**寄存器重命名+指令调度**，避免迭代间新冲突，最大化并行性。

经典例题：向量加法循环优化

源代码：

源代码 C++

```
for(i=1;i<=1000;i++)  x[i]=x[i]+s;
```

MIPS汇编：

MIPS汇编语言 C++

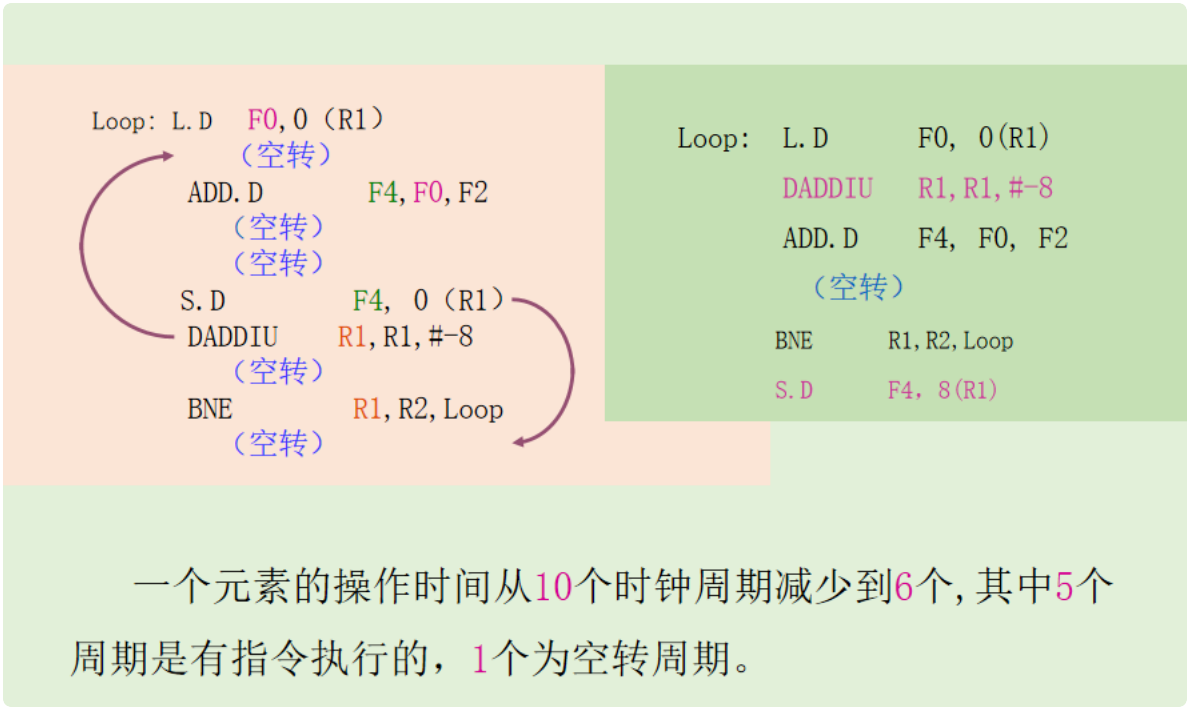
```
Loop:
L.D F0,0(R1)  // 取数
ADD.D F4,F0,F2  // 加法（F2存常数s）
S.D F4,0(R1)    // 存数
DADDIU R1,R1,#-8 // 地址递减
BNE R1,R2,Loop  // 循环判断
```

情况1：无任何调度

			指令流出时钟
Loop:	L.D.	F0, 0 (R1)	1
	(空转)		2
	ADD.D	F4,F0,F2	3
	(空转)		4
	(空转)		5
	S.D	F4, 0 (R1)	6
	DADDIU	R1,R1, # -8	7
	(空转)		8
	BNE	R1,R2,Loop	9
	(空转)		10

每个元素的操作需要10个时钟周期，其中5个是空转周期。

情况2：仅指令调度



一个元素的操作时间从10个时钟周期减少到6个，其中5个周期是有指令执行的，1个为空转周期。

情况3：4次循环展开（无调度）

			指令流出时钟			指令流出时钟
Loop:	L.D	F0,0(R1)	1	ADD.D	F12,F10,F2	15
	（空转）		2	（空转）		16
	ADD.D	F4,F0,F2	3	（空转）		17
	（空转）		4	S.D	F12,-16(R1)	18
	（空转）		5	L.D	F14,-24(R1)	19
	S.D	F4, 0（R1）	6	（空转）		20
	L.D	F6,-8(R1)	7	ADD.D	F16,F14,F2	21
	（空转）		8	（空转）		22
	ADD.D	F8,F6,F2	9	（空转）		23
	（空转）		10	S.D	F16,-24(R1)	24
	（空转）		11	DADDIU	R1,R1,#-32	25
	S.D	F8, -8（R1）	12	（空转）		26
	L.D	F10,-16(R1)	13	BNE	R1,R2,Loop	27
	（空转）		14	（空转）		28

结果分析：

- 这个循环每遍共使用了28个时钟周期。
- 有4个循环体，完成4个元素的操作。

  平均每个元素使用28/4=7个时钟周期
- 原始循环的每个元素需要10个时钟周期。

  节省的时间：从减少循环控制的开销中获得的。
- 在整个展开后的循环中，实际指令只有14条，其他14个周期都是空转。

  效率并不高

情况4：循环展开+指令调度

情况 IV ： 将情况 III 进行指令调度

			指令流出时钟
Loop:	L.D	F0,0(R1)	1
	L.D	F6,-8(R1)	2
	L.D	F10,-16(R1)	3
	L.D	F14,-24(R1)	4
	ADD.D	F4,F0,F2	5
	ADD.D	F8,F6,F2	6
	ADD.D	F12,F10,F2	7
	ADD.D	F16,F14,F2	8
	S.D	F4,0(R1)	9
	S.D	F8,-8(R1)	10
	DADDIU	R1,R1,#-32	11
	S.D	F12,16(R1)	12
	BNE	R1,R2,Loop	13
	S.D	F16,8(R1)	14

结果分析：

- 没有数据相关引起的空转等待。整个循环仅仅使用了14个时钟周期。平均每个元素的操作使用14/4=3.5个时钟周期。
- 通过循环展开、寄存器重命名和指令调度，可以有效地开发出指令级并行。

（注意图中两个红色指令：调度后发生变化）

循环展开与调度六大注意事项

1. 保证正确性：严格修正循环控制条件、**访存偏移量**；

2. 保证有效性：仅对迭代间无依赖的循环展开；

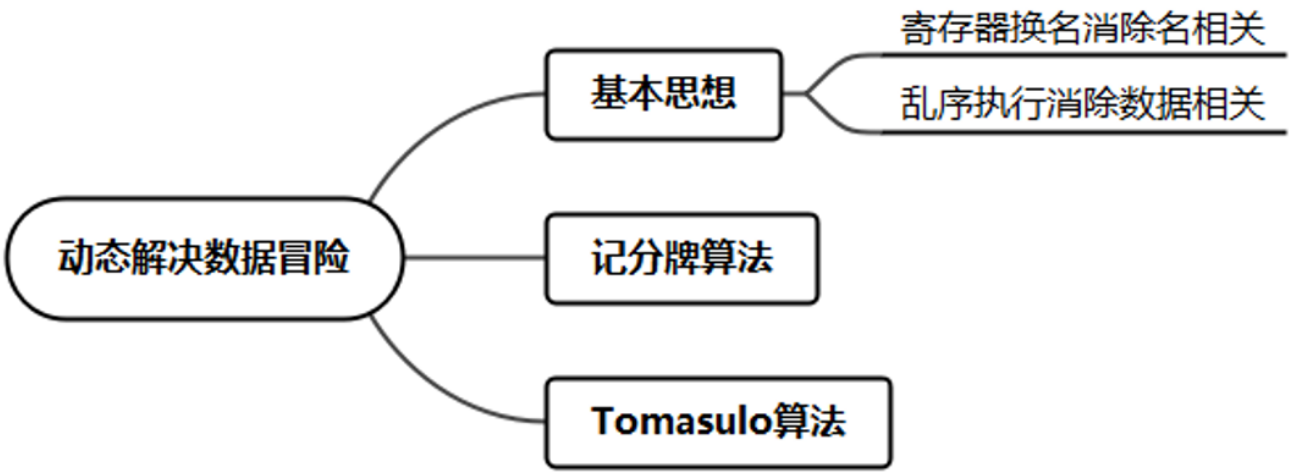
3. 寄存器差异化：不同迭代使用**不同寄存器**，避免新冲突；

4. 精简指令：删除冗余分支、循环判断指令；

5. 规避新相关：展开后多迭代指令共存，需检测新增数据/名相关；

6. 注意对存储器数据的相关性分析例如：对于load指令和store指令，如果它们在不同的循环迭代中访问的存储器地址是不同的，它们就是相互独立的，可以相互对调。

四、动态调度解决数据冒险



4.1 静态调度 vs 动态调度；指令顺序执行 vs 指令乱序执行

调度方式	执行阶段	核心特点
静态调度	编译阶段（软件）	提前调度，无法处理运行时未知相关，适配性差
动态调度	运行阶段（硬件）	实时调度，支持乱序执行，适配运行时动态相关，硬件复杂度高

指令顺序执行：指令放入流水线的顺序和指令完成的顺序一致；

指令乱序执行：指令放入流水线的顺序和指令完成的顺序不一致。

4.2 动态调度核心改造

将**流水线ID段拆分为两个阶段**，支撑乱序执行：

1. **流出（IS）**：指令译码，检测结构冲突，无冲突则送入保留站；

2. **读操作数（RO）**：检测数据冲突，操作数就绪则执行，否则等待。

4.3 动态调度核心问题与解决

1. 问题：顺序指令中，**无关指令会因前序指令阻塞而等待，浪费并行性；**

2. 衍生问题：乱序执行引发**WAR、WAW名相关冲突**；

3. 解决衍生问题方案：**寄存器换名技术，彻底消除名相关。**示例如下：

```
于是下面这段代码：
      指令1    DIV.D   F4，F0，F2
      指令2    SUB.D   F10，F4，F6
      指令3    ADD.D   F6，F12，F14

可以变成：
      指令1    DIV.D   F4，F0，F2
      指令2    SUB.D   F10，F4，F6
      指令3    ADD.D   S，F12，F14
```

4.4 动态调度基本思想小结

对比维度	静态指令调度（编译器处理）	动态指令调度（硬件处理）
调度时机	编译时提前排好指令顺序	**运行时硬件实时调度**
处理能力	只能处理编译时已知的相关	1. 处理**编译时不明的相关**（比如存储器访问冲突，只有运行时才知道两条 store 会不会访问同一地址） 2. 无关指令可先执行，直接减少阻塞停顿
代码适配性	优化后的代码和特定流水线绑定，换流水线需重新编译	同一份编译代码，在不同动态调度流水线上都能高效执行，**可移植性强**

动态调度允许指令乱序执行，会带来两个问题，必须配套解决：

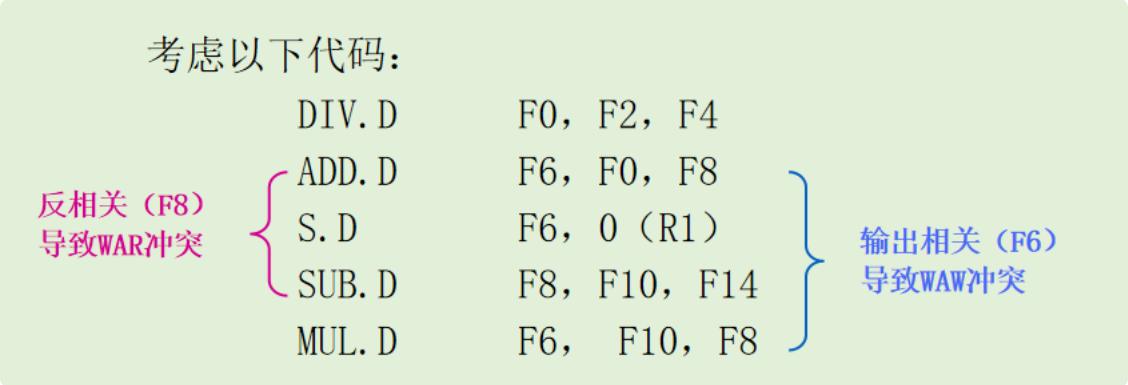
- **名相关（WAR/WAW 假相关）**：不同指令读写同一个寄存器名，乱序执行会导致数据错误，因此必须用**寄存器换名技术**彻底消除假名相关。
- **异常处理困难**：指令乱序完成，程序顺序被打乱，若前面的指令触发异常，后面已执行的指令会留下副作用，难以实现精确异常处理（这也是后续前瞻执行 ROB 所解决的核心问题）。

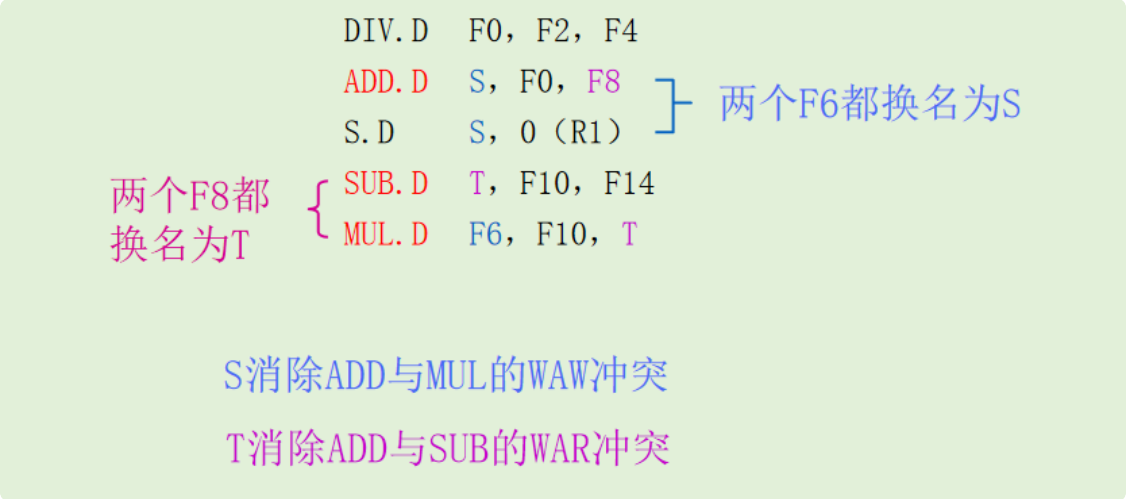
4.4 Tomasulo算法（重中之重，大题核心）

1. 核心思想

1. 分布式冲突检测，操作数就绪立即执行，最小化RAW数据冒险停顿；

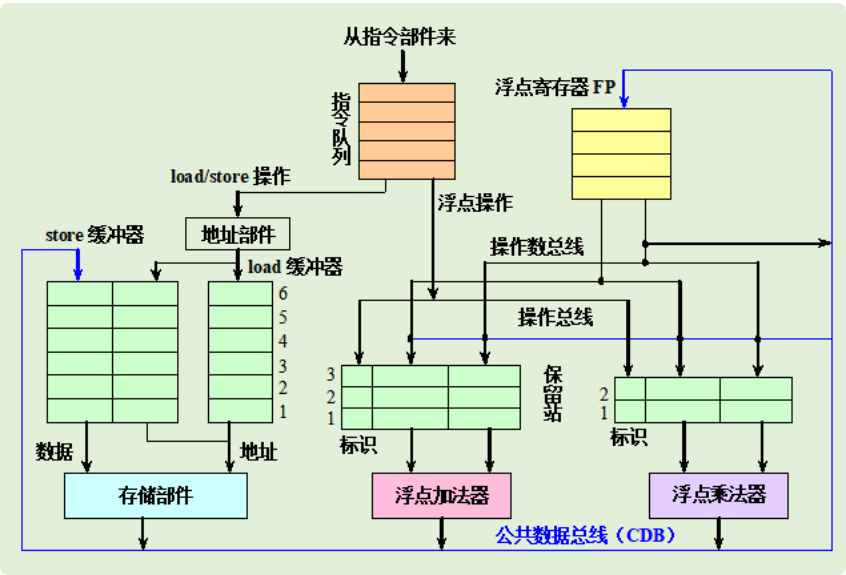
2. 通过保留站实现**硬件寄存器换名**，彻底消除WAR、WAW冲突；示例如下：





3. 结果通过CDB总线广播，直接送达需要的指令，无需经过寄存器。

2. 硬件核心结构



（1）保留站RS：存放已流出、等待执行的指令，记录操作码、操作数、冲突信息；浮点加法3个、乘法2个，负责寄存器换名。

（2）**公共数据总线CDB**：核心数据通路，所有**运算结果、访存结果**通过CDB广播，供所有部件接收。

（在具有多个执行部件且采用多流出（即每个时钟周期流出多条指令）的流水线中，需要采用多条CDB。）

CDB连接到除了load缓冲器以外的所有部件的入口。

（3）**Load/Store缓冲器**：存放访存指令地址、**数据**，**保证访存FIFO顺序**，解决访存冲突。

（4）**寄存器状态表Qi**：记录每个寄存器的结果由哪个保留站产生，为换名提供依据；Qi=0表示数据就绪。

（5）**指令队列、浮点寄存器、运算部件**：负责指令缓存、数据存储、运算执行。

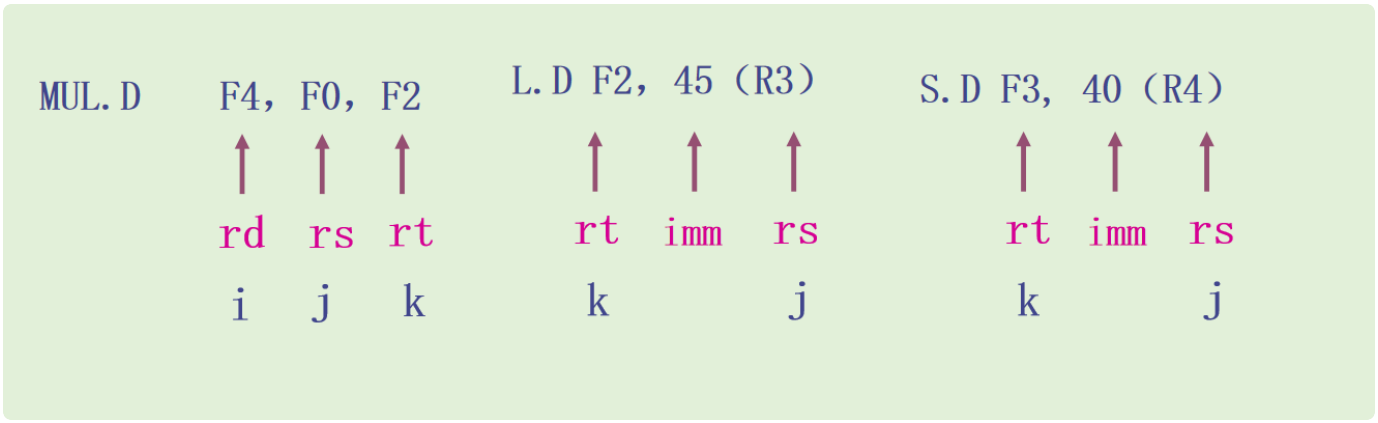
3. 三大执行阶段

符号说明：

符号	核心含义
`r`	分配给当前指令的**保留站 / 缓冲单元编号**，唯一标识硬件资源
`RS`	保留站（浮点运算指令的硬件缓冲）
`imm`	符号扩展后的立即数（访存指令的内存偏移量）
`Regs[]`	寄存器组，保存当前所有寄存器的数值
`Op`	当前指令的操作码（如 `MUL.D` 、 `L.D` ）
`result`	浮点运算部件 / Load 缓冲返回的结果
`Qi`	寄存器状态表，记录「哪个硬件单元会产生该寄存器的下一个值」（寄存器换名核心）

符号	核心含义
`Vj/Vk`	已就绪的操作数值，直接保存寄存器传来的值，无需再访问寄存器
`Qj/Qk`	操作数等待标记，记录「哪个硬件单元会产生该操作数」，用于匹配 CDB 广播

> `Qi/Qj/Qk` 的取值仅表示数据是否就绪，规则统一：
>
> - `= 0` ：数据已就绪
> - `Qi=0` ：寄存器中的数据已就绪
> - `Qj/Qk=0` ：保留站 / 缓冲的 `Vj/Vk` 字段数据已就绪
> - `> 0` （正整数）：数据未就绪，正在等待该编号的硬件单元（保留站 / 缓冲）产生结果



阶段一：流出（Issue）

1. 浮点运算指令（如 `MUL.D F4, F0, F2`）

进入条件：存在空闲的浮点运算保留站（设为 `r`）

步骤	操作逻辑	作用

① 处理第一源操作数rs（如F0）	`if Qi[rs] == 0:`
	`RS[r].Vj = Regs[rs], RS[r].Qj = 0`
	`else:`
	`RS[r].Qj = Qi[rs]`
		就绪时直接复制值；未就绪时记录“谁会产生该操作数”，完成源操作数换名

② 处理第二源操作数rt（如F2）	`if Qi[rt] == 0:`
	`RS[r].Vk = Regs[rt], RS[r].Qk = 0`
	`else:`
	`RS[r].Qk = Qi[rt]`
		同上，处理第二个源操作数的就绪检测与换名

③ 设置保留站状态	`RS[r].Busy = yes, RS[r].Op = 操作码（如MUL.D）`	标记保留站为“忙”，记录指令类型
④ 预约目的寄存器rd（如F4）	`Qi[rd] = r`	消除 WAW 冲突：把当前保留站号写入寄存器状态表，表示“rd的下一个值由RS[r]产生”，后续写同寄存器的指令会覆盖该预约

2. Load 指令（如 `L.D F2, 45(R3)`）

进入条件：存在空闲的 Load 缓冲单元（设为 `r`）

步骤	操作逻辑	作用

① 处理基址寄存器rs（如R3）	`if Qi[rs] == 0:`
	`RS[r].Vj = Regs[rs], RS[r].Qj = 0`
	`else:`
	`RS[r].Qj = Qi[rs]`
		检测基址是否就绪，未就绪时记录产生基址的部件

② 设置缓冲单元状态	`RS[r].Busy = yes, RS[r].A = 符号扩展后的偏移量（如45）`	标记缓冲单元为“忙”，保存访存偏移量
③ 预约目标寄存器rt（如F2）	`Qi[rt] = r`	消除后续写rt的 WAW 冲突，标记“rt的下一个值由 Load 缓冲r产生”

3. Store 指令（如 `S.D F4, 40(R4)`）

进入条件：存在空闲的 Store 缓冲单元（设为 `r`）

步骤	操作逻辑	作用
① 处理基址寄存器rs（如R4）	与 Load 指令基址处理逻辑完全一致	检测基址是否就绪，未就绪时记录产生基址的部件

② 处理待存储数据rt（如F4）	`if Qi[rt] == 0:`
	`RS[r].Vk = Regs[rt], RS[r].Qk = 0`
	`else:`
	`RS[r].Qk = Qi[rt]`
		检测待存数据是否就绪，未就绪时记录产生数据的部件，处理 RAW 相关

③ 设置缓冲单元状态	`RS[r].Busy = yes, RS[r].A =` 符号扩展后的偏移量（如40）	标记缓冲单元为“忙”，保存访存偏移量

阶段二：执行（Execute）

1.1. 浮点操作指令（`ADD.D` / `MUL.D` 等）

进入条件（必须同时满足）

两个源操作数均已就绪：`RS[r].Qj = 0` 且 `RS[r].Qk = 0`

1.2. Load/Store 访存指令

共性前提：所有访存指令需进入**统一的访存缓冲队列**，按程序原始顺序（FIFO）排队，仅队头指令可发起访存操作，避免内存数据冲突。

Load 指令（读内存）

进入条件（必须同时满足）

1. 基址操作数就绪：`RS[r].Qj = 0`（基址寄存器 `rs` 的值已就绪，暂存到 `Vj` 字段）
2. 成为访存缓冲队列的**队头**（前面所有访存指令均已处理完成）

核心操作

1. 计算有效访存地址：`RS[r].A ← RS[r].Vj + RS[r].A`（基址寄存器值 + 偏移量）
2. **访存读数据**：从内存地址 `Mem[RS[r].A]` 读取数据，暂存后准备通过 CDB 广播写回目标寄存器。

Store 指令（写内存）

进入条件（必须同时满足）

1. 基址操作数就绪：`RS[r].Qj = 0`（基址寄存器 `rs` 的值已就绪，暂存到 `Vj` 字段）
2. 成为访存缓冲队列的**队头**（前面所有访存指令均已处理完成）

注意：Store 指令的待存数据（`rt` 寄存器对应操作数）可暂未就绪（`Qk` 无需为 0），执行阶段仅需基址就绪。

核心操作

1. 计算有效访存地址：`RS[r].A ← RS[r].Vj + RS[r].A`（基址寄存器值 + 偏移量）

2. **暂存地址，不立即访存**：访存写内存的操作延迟到「写回阶段」执行，需同时满足「队头 + 待存数据就绪（`Qk=0`）」两个条件才能发起写操作。

阶段三：写结果（Write CDB）

1.1. 浮点运算指令 & Load 指令

进入条件（必须同时满足）

指令执行结束：保留站 / Load 缓冲 `r` 已完成运算 / 访存，产生结果；CDB 就绪：公共数据总线未被占用，可用于广播结果。

核心操作流程（按顺序执行）

广播结果：通过 CDB 向全系统发送「结果值 + 自身硬件编号 `r`」，所有寄存器和保留站监听 CDB；

更新寄存器状态：

- 遍历所有寄存器，若 `Qi[x] = r`（表示寄存器 `x` 正在等待该结果）：
- `Regs[x] ← result`：将结果写入寄存器；
- `Qi[x] ← 0`：标记寄存器值已就绪，消除 WAW 冲突；

1. 遍历所有寄存器，若 `Qi[x] = r`（表示寄存器 `x` 正在等待该结果）：
2. `Regs[x] ← result`：将结果写入寄存器；
- 遍历所有寄存器，若 `Qi[x] = r`（表示寄存器 `x` 正在等待该结果）：
3. `Qi[x] ← 0`：标记寄存器值已就绪，消除 WAW 冲突；

更新等待的保留站：

- 遍历所有保留站，若 `RS[x].Qj = r`（保留站 `x` 的第一操作数正在等待该结果）：
- `RS[x].Vj ← result`：将结果写入保留站的操作数 `Vj` 字段；
- `RS[x].Qj ← 0`：标记该操作数已就绪；
- 同理，若 `RS[x].Qk = r`（保留站 `x` 的第二操作数正在等待该结果）：
- `RS[x].Vk ← result`，`RS[x].Qk ← 0`；

释放硬件资源：`RS[r].Busy ← no`，标记当前保留站 / Load 缓冲为空闲状态，可分配给新指令。

1.2. Store 指令（直接写内存，无 CDB 广播）

进入条件（必须同时满足）

- 指令执行结束：Store 缓冲 `r` 已完成访存地址计算；
- **待存数据就绪：`RS[r].Qk = 0`（表示要写入内存的数据已准备好，存放在 `RS[r].Vk` 字段中）。**

核心操作流程

- **写回内存**：按执行阶段计算好的地址，将待存数据写入内存：`Mem[RS[r].A] ← RS[r].Vk`；
- **释放硬件资源**：`RS[r].Busy ← no`，标记当前 Store 缓冲为空闲状态。

关键特点

- 不更新任何寄存器，也不通过 CDB 广播，因此无需修改寄存器状态表 `Qi`；
- **写内存操作仅需地址和数据就绪，无需等待总线资源。**

总结

	流出	执行	访存	写结果
浮点ALU指令	进入条件：保留站有空闲； 完成任务：进入保留站，换名，目标寄存器预约；	进入条件：两个操作数就绪，且部件不冲突； 完成任务：浮点运算		进入条件：功能部件执行结束，CDB就绪； 完成任务：将结果通过CDB写入浮点寄存器和保留站，修改Qi，Qj，Qk状态，释放保留站；
Load指令	进入条件：L/S缓冲区有空闲； 完成任务：进入缓冲区，立即数写入A字段；	进入条件：L/S队列头部，rs寄存器就绪，计算部件不冲突； 完成任务：计算访存地址，并写入A字段；	进入条件：地址已经计算完，写入A字段； 完成任务：访存，结果放到CDB上；	进入条件：访存结束，CDB就绪； 完成任务：将结果通过CDB写入浮点寄存器和保留站，修改Qi，Qj，Qk状态，释放缓冲区；
Store指令	进入条件：L/S缓冲区有空闲； 完成任务：进入缓冲区，立即数写入A字段；	进入条件：L/S队列头部，rs寄存器就绪，计算部件不冲突； 完成任务：计算访存地址，并写入A字段；	进入条件：地址已经计算完，写入A字段，要写入的数据就绪； 完成任务：写入存储器；	这个访存就是它的写结果阶段

注意：还有一个隐含条件：没有结构冲突

4. 算法优缺点

优点：

1. 冲突检测和指令执行控制是「分布的」

- 每个保留站自己判断“能不能执行”
- 结果不用走寄存器，直接通过 CDB “广播分发”

2. 消除了 WAW 和 WAR 冲突导致的停顿

缺点：硬件复杂度高、**无法处理控制冒险和程序异常**。

5. 核心局限

仅解决结构冲突、数据冲突；无分支预测、无异常处理能力，需结合前瞻执行技术优化。

指令队列里若有分支指令，分支指令之后的指令不能流出，直到确定分支是否成功。

6.一个简单例子

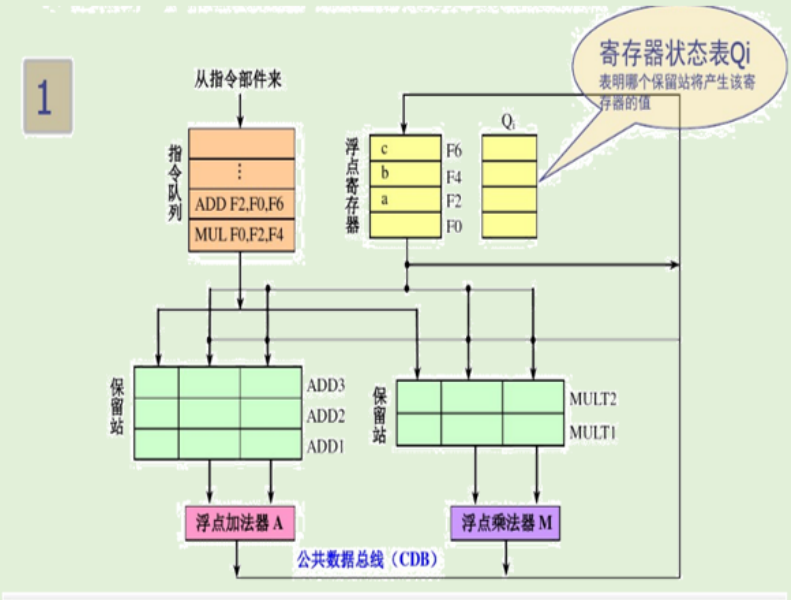


图3.2(a)

MUL F0,F2,F4
ADD F2,F0,F6

把图3.1简化为图3.2(a)，并且增加了一个寄存器状态表Q_i。每个寄存器在Q_i中有一项，用于指出哪个保留站将产生该寄存器的值。

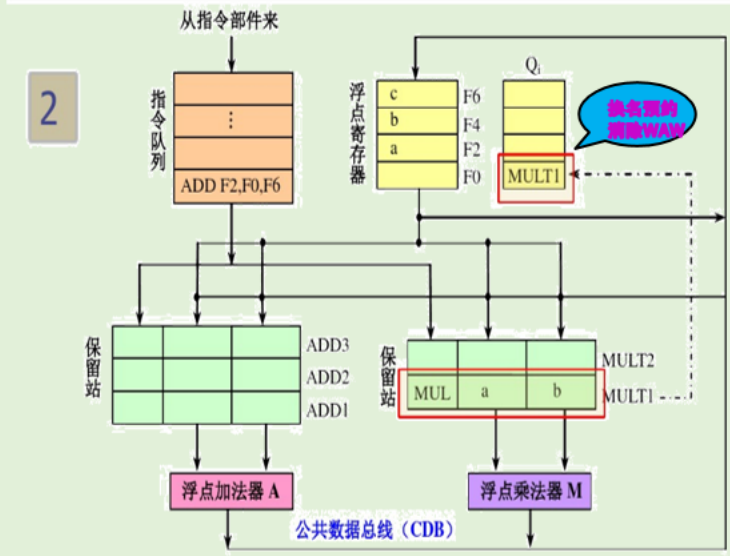


图3.2(b)

MUL F0,F2,F4
ADD F2,F0,F6

图3.2(b)，当指令MULF0, F2, F4流出到保留站 Mult1时，由于其操作数a和b就绪(在F2和F4中)，就将它们从寄存器取到保留站，这样该指令以后就跟F2和F4没有关系了，执行时直接从保留站中取数据。同时将目的寄存器F0对应的Q_i标志置为Mut1，表示该寄存器的内容将由保留站 Mult1提供(图3.2(b)中的虚线所示)。

解释换名预约消除WAW：本例中无法体现

例子：MUL F0, F2, F4 ;

ADD F0, F1, F3 ;

执行完第一条指令后，F0对应的Qi处是MULT1；执行完第二条指令后，会把F0对应的Qi更新为ADD1;

- 当 `Mult1` 执行完成，把结果发到公共数据总线（CDB）时，硬件会检查：哪些寄存器的 `Qi` 等于 `Mult1`？此时 `F0` 的 `Qi` 已经被改成 `ADD1` 了，所以 `F0` 不会接收 `Mult1` 的结果，`Mult1` 的结果只会被那些还在等待它的指令接收（如果有的话），不会写入 `F0` 寄存器。
- 当 `ADD1` 执行完成，把结果发到 CDB 时，`F0` 的 `Qi` 等于 `ADD1`，所以 `F0` 会接收这个结果，写入寄存器。

这样，即使指令 2（`ADD`）比指令 1（`MUL`）先执行完成，也不会出错：`ADD` 的结果会被写入 `F0`，而 `MUL` 的结果不会再写入 `F0`，完全避免了 WAW 冲突！

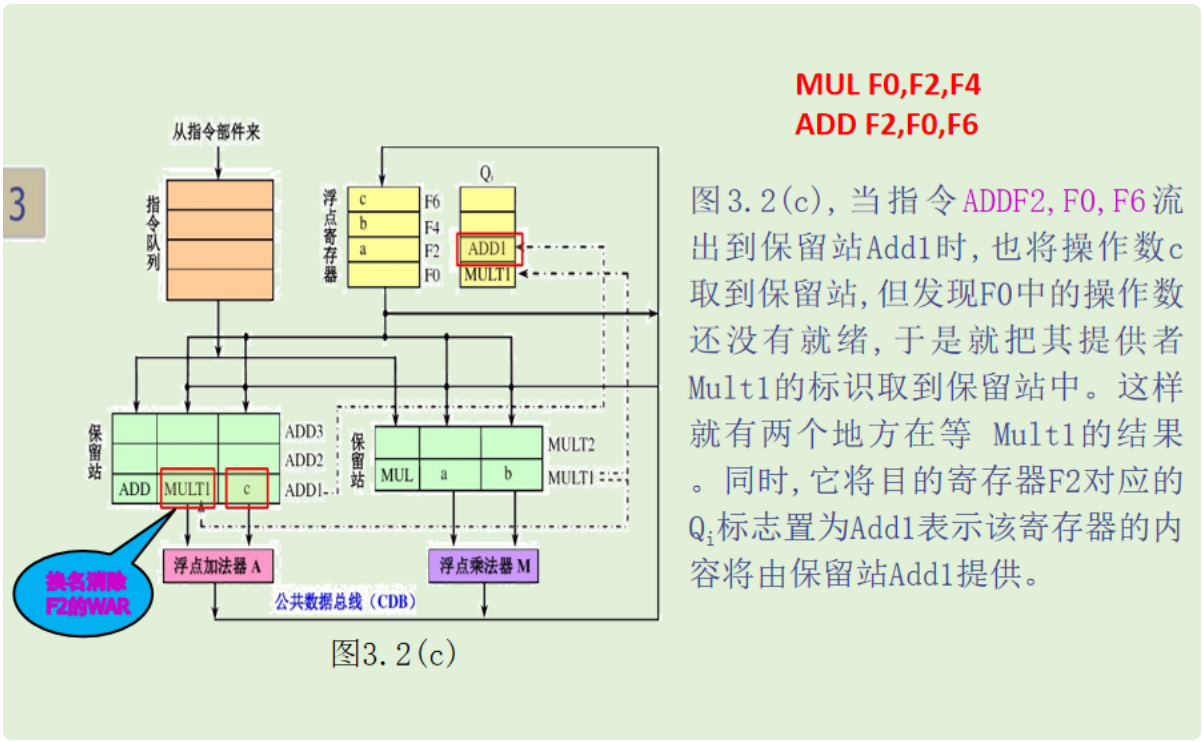


图3.2(c)

MUL F0,F2,F4
ADD F2,F0,F6

图3.2(c)，当指令ADDF2,F0,F6流出到保留站Add1时，也将操作数c取到保留站，但发现F0中的操作数还没有就绪，于是就把其提供者Mult1的标识取到保留站中。这样就有两个地方在等 Mult1的结果。同时，它将目的寄存器F2对应的Q_i标志置为Add1表示该寄存器的内容将由保留站Add1提供。

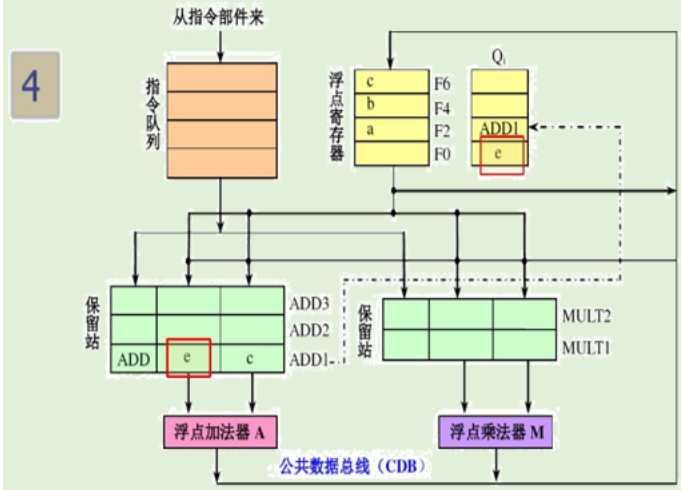


图3.2(d)

MUL F0,F2,F4
ADD F2,F0,F6

图3.2(d)，当 Mult1的运算结果产生后(设为e)，就把数据放到总线上(广播)，所有等待该数据的地方都会自动把数据取走。Add1中的ADD指令得到该数据后，马上就可以开始执行。

> 这里e的位置错误：应该在浮点寄存器中，而不是Qi中。
>
> - 把 CDB 上的 `e` 写入 F0 寄存器；
> - 同时把 `Qi` 标记清掉，表示“F0 的值已经更新完成”。

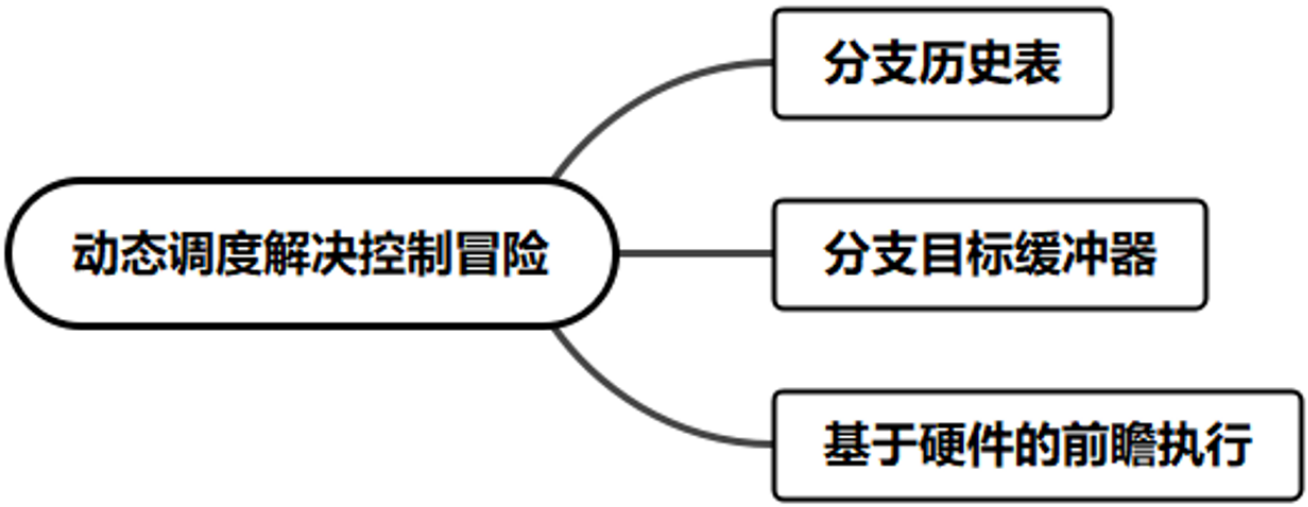
五、动态调度解决控制冒险

5.1 五大要点

静态分支预测 vs 动态分支预测

1. 静态预测：**编译时预设预测规则**，与程序运行状态无关，适应性差、准确率低；

2. 动态预测：**硬件**实时记录分支历史，根据运行状态动态调整预测结果，准确率高、适应性强。



什么是“分支指令过去的表现”？

就是记录分支的历史信息，在预测错误时，要作废已经预取和分析的指令，恢复现场，为了恢复现场，需要在执行预测的目标指令之前将现场保存起来。

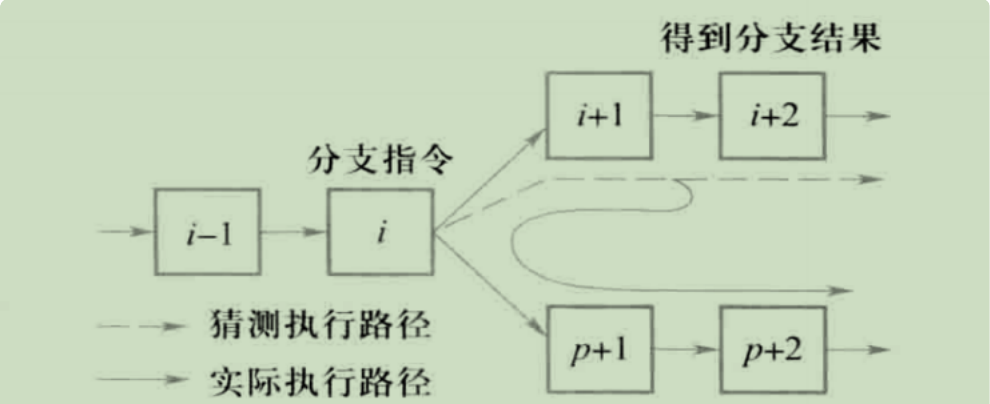


图 4.5　分支预测不成功时的重新执行过程

https://blog.csdn.net/weixin_43633784

分支预测的有效性取决于?

预测的准确性；

预测正确和不正确两种情况下的分支开销；

决定分支开销的因素：

- 流水线的结构（5段流水，分支处理由MEM段提到ID）
- 预测的方法（静态or动态）
- 预测错误时的恢复策略等

采用动态分支预测技术的目的

预测分支是否成功；

尽快找到分支目标地址（或指令）（避免控制相关造成流水线停顿）。

需要解决的关键问题

如何记录分支的历史信息；

如何根据这些分支的历史信息来预测分支的去向（甚至取到指令）。

5.2 分支历史表 BHT（1位/2位预测）【分支预测缓冲器】

1. 基本原理

用硬件缓存记录分支指令的历史执行结果（跳转/不跳转），依据历史预测当前分支走向，是最简单的动态分支预测技术。

2. 1位预测器（记录最近一次的历史）

状态：0（分支不跳转）、1（分支跳转）；

规则：预测错误则翻转状态，预测正确则保持；

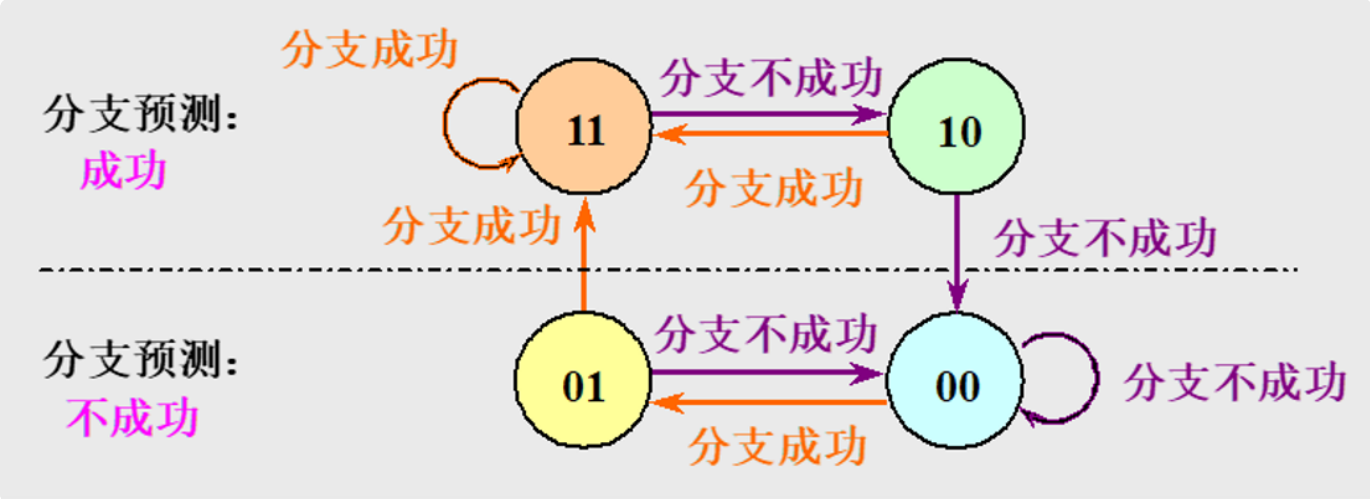
缺陷：单次错误就翻转，抖动性强，准确率一般。

预测时的2个步骤：
分支预测+预测位修改（置1或置0）；

分支预测错误时，预测位就被修改，并且需要恢复现场，程序从分支指令处重新执行。

3. 2位预测器（主流）

四个状态：00、01（预测不跳转）、10、11（预测跳转）；



（图中的成功/不成功指是否跳转）

核心规则：**连续两次预测错误才翻转预测方向**，抗抖动、准确率高；

结论：2位预测性能与n位（n>2）接近，性价比最高，工业界通用4K BHT。

4. BHT适用场景

仅适用于：分支判定延迟 > 分支目标地址计算延迟的流水线；

经典5段流水线无优化效果。

> **为什么“判定时间晚于地址计算时间”时，BHT 才有用?**
>
> 流水线的核心瓶颈在**取指阶段（IF）**：取指必须知道下一条指令的地址，才能提前取指、避免停顿。
>
> - 如果“判定分支是否成功”的时间比“计算目标地址”晚，意味着：
>   a. 目标地址早就算好了（比如在 ID 段就完成了），
>   b. 但分支到底跳不跳，要等后面的阶段（比如 EX 段）才能知道。这时候，IF 段取指的时候，我们还不知道要不要跳转，只能先取顺序指令，等后面阶段知道结果后，如果取错了，就要清空流水线，产生 **2 个周期的停顿**。
> - 这时候 BHT 就能发挥作用：在 IF 段就根据历史预测跳不跳，提前取正确的地址，直接消除停顿。

> **为什么经典 5 段流水用 BHT 没用?**
>
> PPT 里说：“经典 5 段流水里，判定分支是否成功和计算目标地址都在 ID 段完成，所以 BHT 不会带来好处。”这是因为经典 5 段流水的分支处理逻辑非常紧凑：
>
> - 计算分支目标地址：**ID 段完成**（PC + 偏移量，用 ID 段的加法器就能算）
> - 判定分支是否成功：**ID 段完成**（读寄存器值、比较条件，ID 段刚好能拿到寄存器的值）
>
> 也就是说，在 ID 段的时候，我们已经**同时知道了“跳不跳”和“跳去哪”**，下一条指令的地址已经确定了。这时候 IF 段取完当前分支指令，ID 段就直接给出了下一条指令的地址，根本不需要提前预测，BHT 的预测功能在这里没有任何价值，所以不会减少停顿。

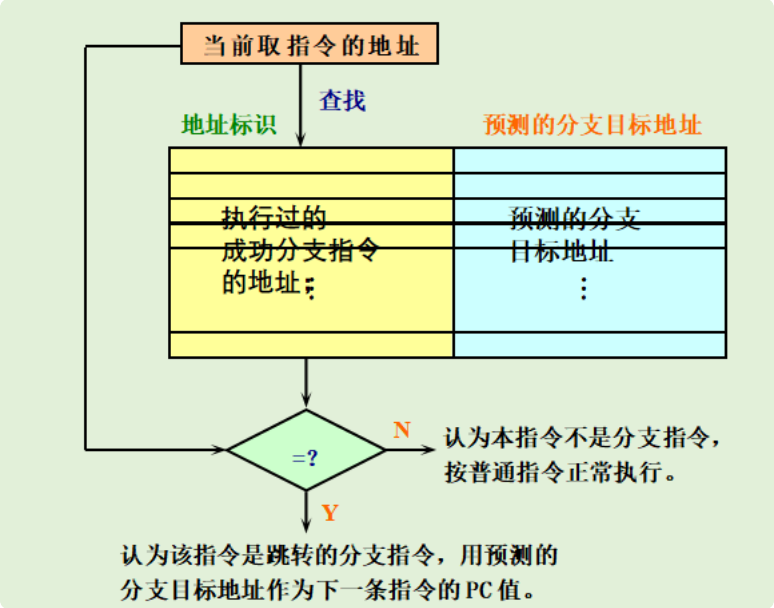
5.3 分支目标缓冲器 BTB

1. 作用

不仅预测分支是否跳转，还**提前缓存分支目标地址/指令**，解决高性能多流出流水线的指令流供给问题。

2. 表结构

核心字段：**上次跳转的分支指令地址（匹配标识）**、预测的分支目标地址。



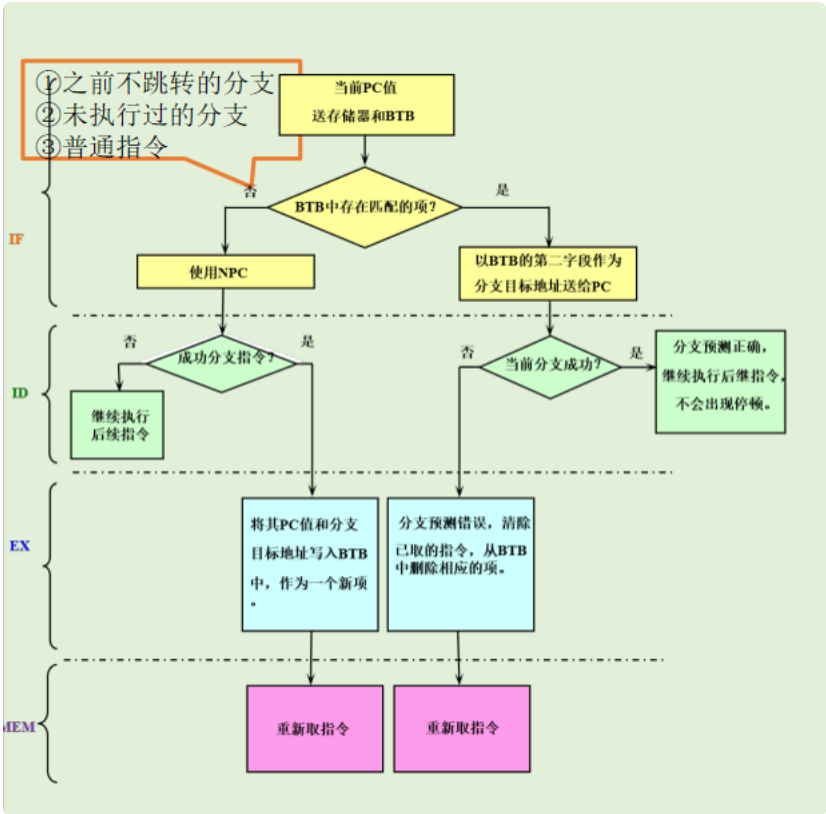
3. 流水线延迟规则

1. BTB命中+预测正确：0停顿周期；

2. BTB命中+预测错误：2周期以上开销；

3. BTB未命中：2周期以上开销。

2/3需要2周期以上开销的原因：① 需要更新BTB中的项，要花费一个时钟周期；② 在更新BTB 中的项时，要停止取指令，那么取新的指令又要花费一个时钟周期。



4. BTB扩展优化

. BTB的另一种形式

在分支目标缓冲器中存放一条或者多条分支目标处的指令。

- 有三个潜在的好处：
  - 更快地获得分支目标处的指令；
  - 可以一次提供分支目标处的多条指令，这对于多流出处理器是很有必要的；
  - 使我们可以进行称为分支折叠（branch folding）的优化。
    实现零延迟无条件分支，甚至有时还可以做到零延迟条件分支。



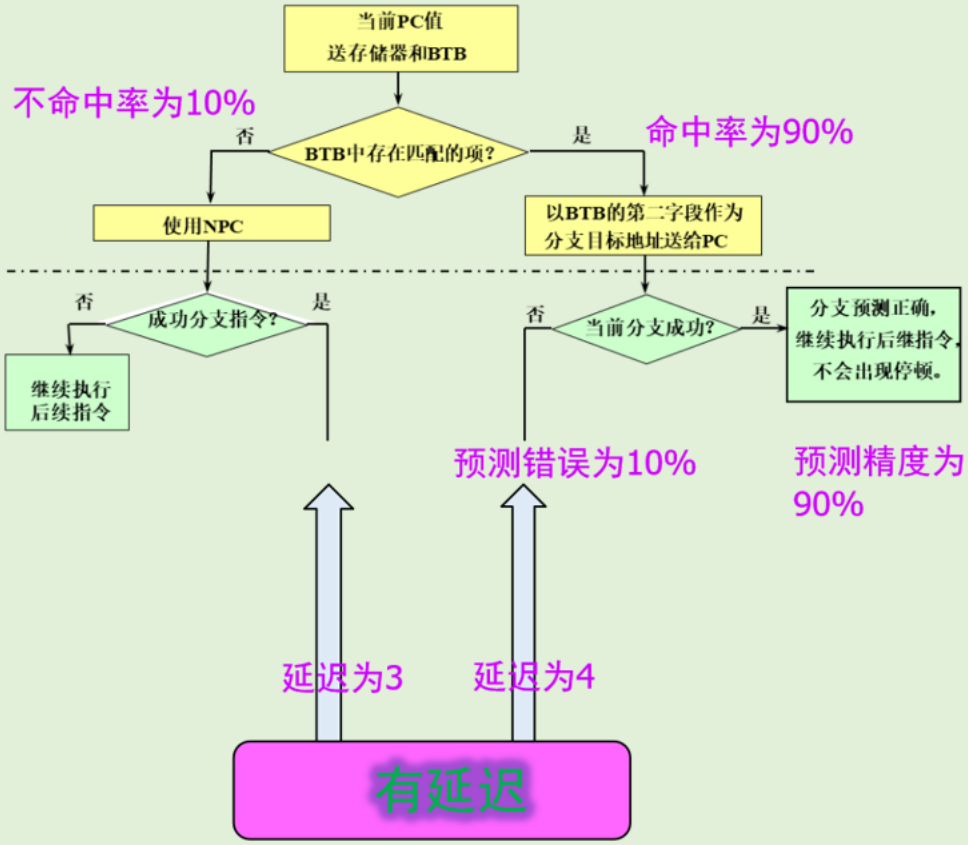
分支指令地址	分支历史表 BHT	分支目标处的若干条指令
A0	T0	$I_{0,0}$, $I_{0,1}$, …, $I_{0,m-1}$
A1	T1	$I_{1,0}$, $I_{1,1}$, …, $I_{1,m-1}$
⋮	⋮	⋮
Ak-1	Tk-1	$I_{k-1,0}$, $I_{k-1,1}$, …, $I_{k-1,m-1}$

5. BTB的CPI计算题

假设有一条长流水线，仅仅对条件转移指令使用分支目标缓冲。假设分支预测错误的开销为4个时钟周期，缓冲不命中的开销为3个时钟周期。假设：命中率为90%，预测精度为90%，分支频率为15%，没有分支的基本CPI为1。

（1）求程序执行的CPI。

（2）相对于采用固定的2个时钟周期延迟的分支处理，哪种方法程序执行速度更快？



解：

（1）程序执行的CPI = 没有分支的基本CPI（1） + 分支带来的额外开销

分支带来的额外开销是指在分支指令中，缓冲命中但预测错误带来的开销与缓冲没有命中带来的开销之和。

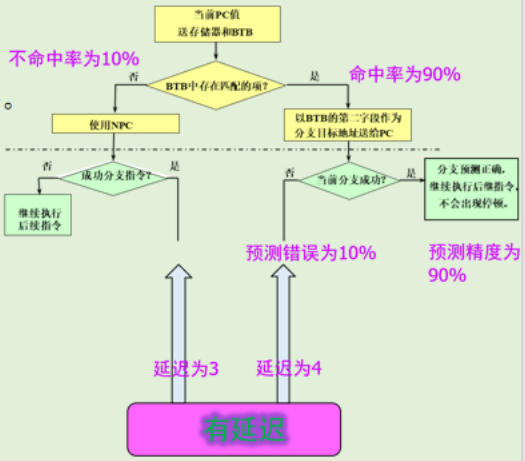
分支带来的额外开销= 15% * (90%命中×10%预测错误×4 + 10%没命中×3)= 0.099

所以，程序执行的CPI = 1 + 0.099 = 1.099

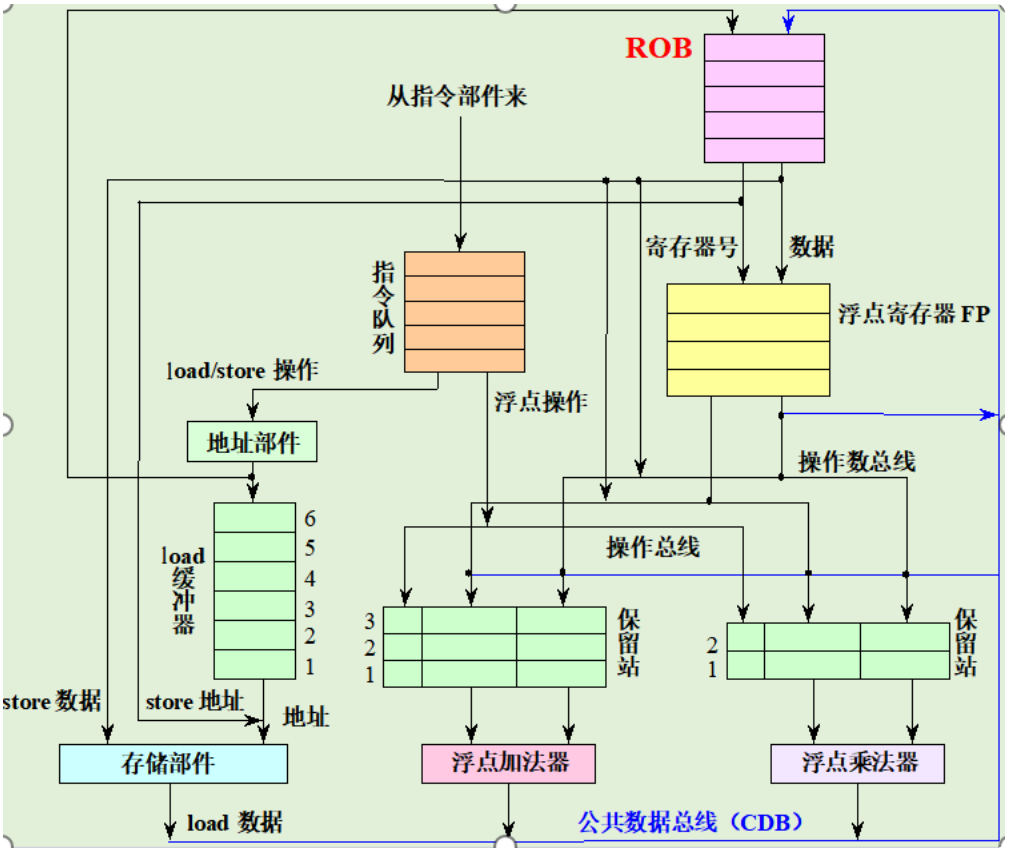
（2）采用固定的2 个时钟周期延迟的分支处理

CPI = 1 + 15%×2 = 1.3

由（1）（2）可知分支目标缓冲方法执行速度快。



5.4 基于硬件的前瞻执行（ROB技术）



1. 设计初衷

弥补Tomasulo算法缺陷：解决**控制冒险、程序异常处理**问题，结合动态分支预测+乱序执行+顺序确认，最大化ILP。

2. 核心思想：动态预测+猜测执行+保留站技术

- 分支预测**猜测执行**后续指令，指令结果不直接写入寄存器/存储器，暂存于ROB重排序缓存，等到相应的指令得到“确认”（commit）（即确实是应该执行的）之后，才将结果写入寄存器或存储器。
- **顺序流出、乱序执行、顺序确认**；
- 预测正确：ROB结果正式写入硬件，完成执行；
- 预测错误/异常：清空ROB、刷新指令队列，回滚现场，无副作用。【没有进行不可恢复的写操作】

  若是异常指令，此时它会到达ROB头部

基于硬件的前瞻执行结合了三种思想：

① 动态分支预测。用来选择后续执行的指令。

② 在控制相关的结果尚未出来之前，前瞻地执行后续指令。

③ 用动态调度对基本块的各种组合进行跨基本块的调度。

（基本程序块：如果 一串连续的代码除了入口和出口以外，不包含其他分支指令和转入点。）

实质是数据流执行(data flow execution)：只要操作数有效，指令就执行

3. ROB重排序缓存结构

每项包含4个核心字段：

- **指令类型**：指出该指令是分支指令、store指令或寄存器操作指令（包括 ALU 运算指令 + Load 指令）；
- **目标地址**：给出指令执行结果应写入的目标寄存器号（如果是load和ALU指令）或存储器单元的地址（如果是store指令）；
- **数据值**：前瞻执行的临时结果；

    结果**不能立刻正式写入寄存器 / 内存**（怕分支预测错误白执行）

    先把运算结果、读到的数据，临时存在这个字段里

    等到确认这条指令确实有效、前面指令都没问题，再把这个值正式提交写进去

- **就绪状态**：一位状态标记，判断指令有没有跑完

    0 = 未就绪：指令还在执行，结果还没算好

    1 = 已就绪：指令已经执行完毕，数据值字段里已经存好结果，只差按顺序提交

4. 前瞻执行与Tomasulo的区别

Tomasulo算法的写结果和指令完成都在“写结果”段完成，而在前瞻执行中加以区分，**分成两个段**：

①写结果段

把前瞻执行的结果写到ROB中；

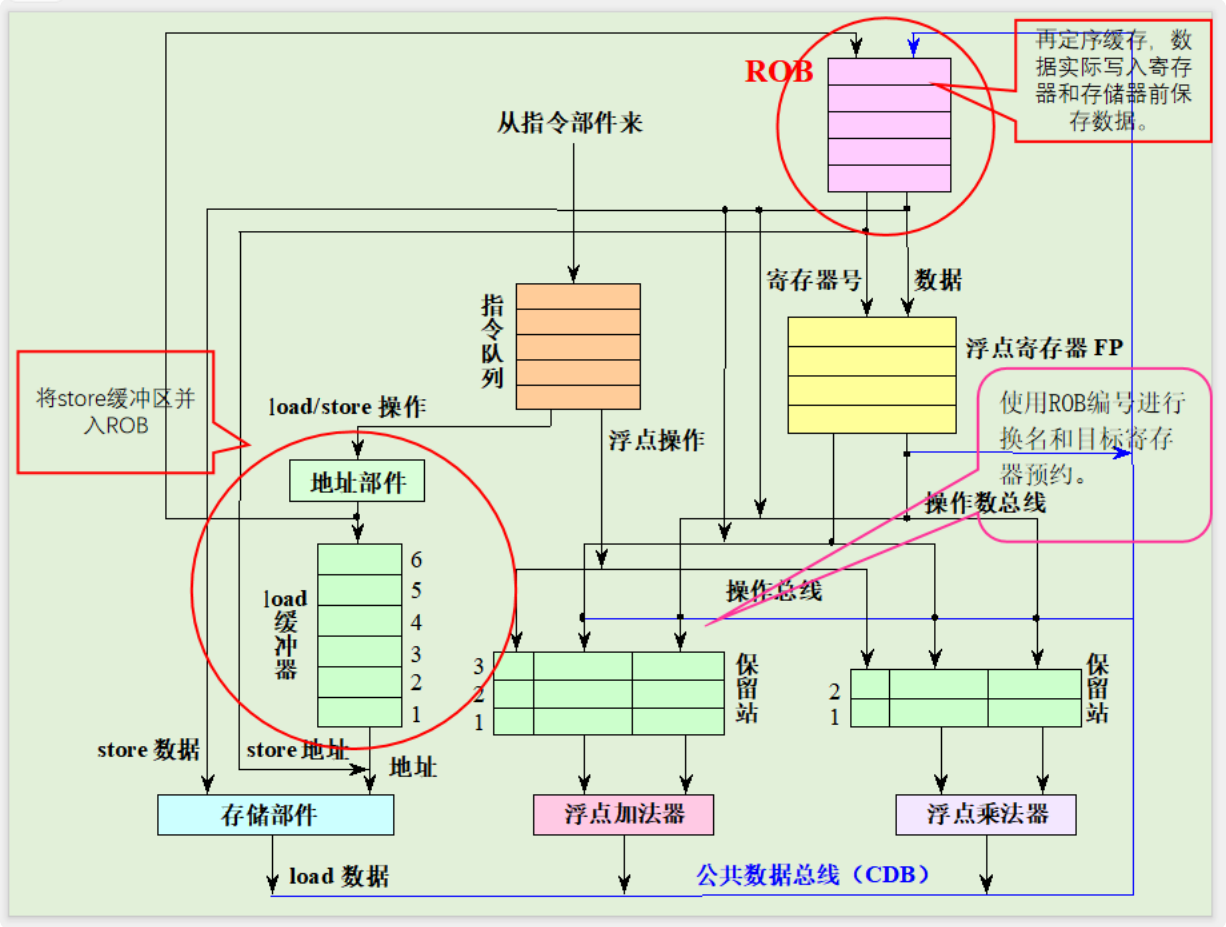
通过CDB在指令之间传送结果，供需要用到这些结果的指令使用。

②指令确认段

在分支指令的结果出来后，对相应指令的前瞻执行给予确认。

如果前面所做的猜测是对的，把在ROB中的结果写到寄存器或存储器。

如果发现前面对分支结果的猜测是错误的，那就不予以确认，并从那条分支指令的另一条路径开始重新执行。



前瞻执行使用ROB编号进行换名和目标寄存器预约

传统 Tomasulo 用“寄存器状态表 + 保留站号”换名

5. 前瞻执行四阶段流程

A. 流出：

流出

- 从浮点指令队列的头部取一条指令。
- 如果有空闲的保留站（设为r）且有空闲的ROB项（设为b），就流出该指令。
- 如果该指令需要的操作数在寄存器或者ROB中就绪，就将其值取到保留站r中；否则，将产生其值的ROB编号写入。
- ROB的编号写入r。
- 用对应的ROB编号对目标寄存器进行预约。

流出（Tomasulo算法）

- 从指令队列的头部取一条指令。
- 如果该指令的操作所要求的保留站有空闲的，就把该指令送到该保留站r。
- 如果该指令需要的操作数在寄存器中就绪，就将值取到保留站r中；如果没就绪，将产生该操作数的保留站的标识送到保留站r。
- 完成对目标寄存器的预约工作。

B. 执行：

执行

- 如果有操作数尚未就绪，就等待，并不断地监测CDB。
- 当两个操作数都已在保留站中就绪后，就可以执行该指令的操作。
- Load指令做有效地址运算和读取数据；store指令只做有效地址运算。

执行（Tomasulo算法）

- 如果有操作数尚未就绪，就等待，并不断地监测CDB。
- 当两个操作数都已在保留站中就绪后，就可以执行该指令的操作。
- Load指令做有效地址运算和读取数据；store指令只做有效地址运算。

C. 写结果：

写结果

- 当结果产生后，将该结果连同本指令在流出段所分配到的ROB项的编号放到CDB上，经CDB写到ROB以及所有等待该结果的保留站。
- 释放产生该结果的保留站。
- store指令在本阶段完成，其操作为:
  - 如果要写入存储器的数据已经就绪，就把该数据写入分配给该store指令的ROB项。
  - 否则，就监测CDB，直到那个数据在CDB上播送出来，这时才将之写入分配给该store指令的ROB项。

写结果（tomasulo算法）

- 当结果产生后，将该结果放到CDB上，经CDB写到所有等待该结果的寄存器和保留站中。
- 当写入地址和数据齐备时，Store指令完成存储器的写入。
- 释放产生该结果的保留站。

D. 确认（独有阶段）：

确认 （Tomasulo算法无该阶段）

对分支指令、**store**指令以及其他指令的处理不同:

- 其他指令（除分支指令和store指令）

  当该指令到达ROB队列的头部而且其结果已经就绪时，就把该结果写入该指令的目标寄存器，并从ROB中删除该指令。

- store指令

  处理与上面类似，只是它把结果写入存储器。

- 分支指令
  - 当预测错误的分支指令到达ROB队列的头部时，清空ROB，并从分支指令的另一个分支重新开始执行。（错误的前瞻执行）
  - 当预测正确的分支指令到达ROB队列的头部时，该指令执行完毕。
- 异常指令（实现精确异常）

  在该指令到达ROB头部前不处理，到达ROB头部时，产生中断，清空ROB，此操作能够完成精确的异常处理。但凡刷新ROB，连同指令队列一起刷新，因为假如指令队列中有指令已经是有问题的指令流。

注意store指令：在写结果阶段是把要写入存储器的数据写入ROB项；在确认阶段是将数据写入存储器。

WtCDB–>Commit–>Mem

6. 核心优缺点

优点：支持动态调度、分支前瞻执行、实现精确异常处理、可拓展至整数部件；

缺点：硬件结构极度复杂，硬件成本高。